

Федеральное государственное автономное образовательное учреждение высшего образования «**Национальный исследовательский университет ИТМО**»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №4
по дисциплине «**Функциональное программирование**»

Аппликативный парсер комбинатор

Преподаватель:
Доморацкий Эридан Алексеевич

Выполнил: Беля Алексей
Группа: P3317

1. Задание

Написать парсер комбинатор на OCaml.

Общие требования:

- программа должна быть реализована в функциональном стиле;
- требуется использовать идиоматичный для технологии стиль программирования;
- задание и коллектив должны быть согласованы;
- допустима совместная работа над одним заданием.

Содержание отчёта:

- титульный лист;
- требования к разработанному ПО, включая описание алгоритма;
- реализация с минимальными комментариями;
- ввод/вывод программы;
- выводы (отзыв об использованных приёмах программирования).

2. Ключевые элементы реализации

Базовый пример

```
open Parser_combinator.Parser

(* Парсер для одиночного символа *)
let p = char_p 'a'

(* Парсинг строки *)
let result = parse_string p "abc"

(* result = Some 'a' *)

(* Комбинирование парсеров *)
let digit = satisfy (fun c -> c >= '0' && c <= '9')
let digits = some digit
let number = fmap (fun chars ->
  int_of_string (String.of_seq (List.to_seq chars))
) digits
```

Аппликативный стиль

```
open Parser_combinator.Parser

(* Парсер для пары координат: (x, y) *)
type point = { x : int; y : int }

let point_parser =
  let make_point x y = { x; y } in
  make_point
    <$> (char_p '(' *> natural)
    <*> (string_p ", " *> natural <* char_p ')')
```

```
let result = parse_string point_parser "(10, 20)"  
(* result = Some { x = 10; y = 20 } *)
```

Альтернативы

```
open Parser_combinator.Parser
```

```
(* Парсер для булевых значений *)
```

```
let bool_parser =  
    (true <$ string_p "true") <|> (false <$ string_p "false")
```

```
let result1 = parse_string bool_parser "true"    (* Some true *)  
let result2 = parse_string bool_parser "false"   (* Some false *)  
let result3 = parse_string bool_parser "maybe" (* None *)
```

Пример парсера арифметических выражений

Библиотека включает полный пример парсера арифметических выражений, следующего грамматике из статьи:

```
expr      ::= constExpr | binOpExpr | negExpr  
const     ::= int  
int       ::= digit{digit}  
digit     ::= '0' | ... | '9'  
binOpExpr ::= '(' expr ' ' binOp ' ' expr ')'  
binOp     ::= '+' | '*'  
negExpr   ::= '-' expr  
open Parser_combinator.Expr_parser
```

```
(* Парсинг и вычисление выражения *)
```

```
let result = parse_and_eval "(1 + ((2 + 3) * (4 + 5)))"  
(* result = Some 46 *)
```

(* Парсинг в AST *)

```
let ast = parse "(1 + 2)"
```

```
(* ast = Some (BinaryExpr (ConstExpr 1, Add, ConstExpr 2)) *)
```

3. Результаты работы тестов

```
opam exec -- dune build && opam exec -- dune runtest
```

Testing `Parser Combinator'.

[OK]	Basic Parsers	0	satisfy success.
[OK]	Basic Parsers	1	satisfy failure.
[OK]	Basic Parsers	2	satisfy empty.
[OK]	Basic Parsers	3	char_p success.
[OK]	Basic Parsers	4	char_p failure.
[OK]	Basic Parsers	5	string_p success.
[OK]	Basic Parsers	6	string_p failure.
[OK]	Basic Parsers	7	string_p partial.
[OK]	Basic Parsers	8	pure.
[OK]	Basic Parsers	9	empty.
[OK]	Basic Parsers	10	eof at end.
[OK]	Basic Parsers	11	eof not at end.
[OK]	Functor	0	fmap.
[OK]	Functor	1	<\$> operator.
[OK]	Functor	2	<\$ replace left.
[OK]	Functor	3	\$> replace right.
[OK]	Applicative	0	apply.

[OK]	Applicative	1	apply sequence.
[OK]	Applicative	2	*> sequence right.
[OK]	Applicative	3	<* sequence left.
[OK]	Applicative	4	lift2.
[OK]	Applicative	5	lift3.
[OK]	Alternative	0	alt first.
[OK]	Alternative	1	alt second.
[OK]	Alternative	2	alt both fail.
[OK]	Alternative	3	many some.
[OK]	Alternative	4	many none.
[OK]	Alternative	5	some success.
[OK]	Alternative	6	some failure.
[OK]	Alternative	7	optional some.
[OK]	Alternative	8	optional none.
[OK]	String Parsers	0	skip_string.
[OK]	String Parsers	1	skip_while.
[OK]	String Parsers	2	take_while.
[OK]	String Parsers	3	take_while1 success.
[OK]	String Parsers	4	take_while1 failure.
[OK]	String Parsers	5	skip_spaces.
[OK]	Character Classes	0	digit.
[OK]	Character Classes	1	letter.
[OK]	Character Classes	2	alphanum.
[OK]	Numeric Parsers	0	natural.
[OK]	Numeric Parsers	1	natural single.
[OK]	Numeric Parsers	2	integer positive.

[OK]	Numeric Parsers	3 integer negative.
[OK]	Combinators	0 between.
[OK]	Combinators	1 sep_by.
[OK]	Combinators	2 sep_by empty.
[OK]	Combinators	3 sep_by1.
[OK]	Combinators	4 choice.
[OK]	Combinators	5 count.
[OK]	Combinators	6 one_of.
[OK]	Combinators	7 none_of.
[OK]	Expression Parser	0 parse constant.
[OK]	Expression Parser	1 parse negation.
[OK]	Expression Parser	2 parse addition.
[OK]	Expression Parser	3 parse multiplication.
[OK]	Expression Parser	4 parse nested.
[OK]	Expression Parser	5 parse double negation.
[OK]	Expression Parser	6 parse negated binary.
[OK]	Expression Parser	7 eval constant.
[OK]	Expression Parser	8 eval negation.
[OK]	Expression Parser	9 eval addition.
[OK]	Expression Parser	10 eval multiplication.
[OK]	Expression Parser	11 eval nested.
[OK]	Expression Parser	12 invalid: leading/trailing spaces.
[OK]	Expression Parser	13 invalid: no parentheses.
[OK]	Expression Parser	14 invalid: double space.

Test Successful in 0.004s. 67 tests run.

4. Листинг программы

https://github.com/aeshabb/lab4_fp

5. Вывод

В ходе лабораторной работы была реализована на OCaml библиотека аппликативных парсер-комбинаторов по мотивам статьи с Habr, спроектирован и реализован парсер арифметических выражений (включая вычисление AST), написан полноценный набор модульных тестов на Alcotest, настроены автоматическая сборка, тестирование, форматирование и линтинг в CI GitHub Actions, а также оформлена документация и русскоязычные комментарии в коде; итоговая система демонстрирует идиоматичный функциональный стиль и показывает, как на практике применять аппликативные и комбинаторные подходы к построению парсеров.